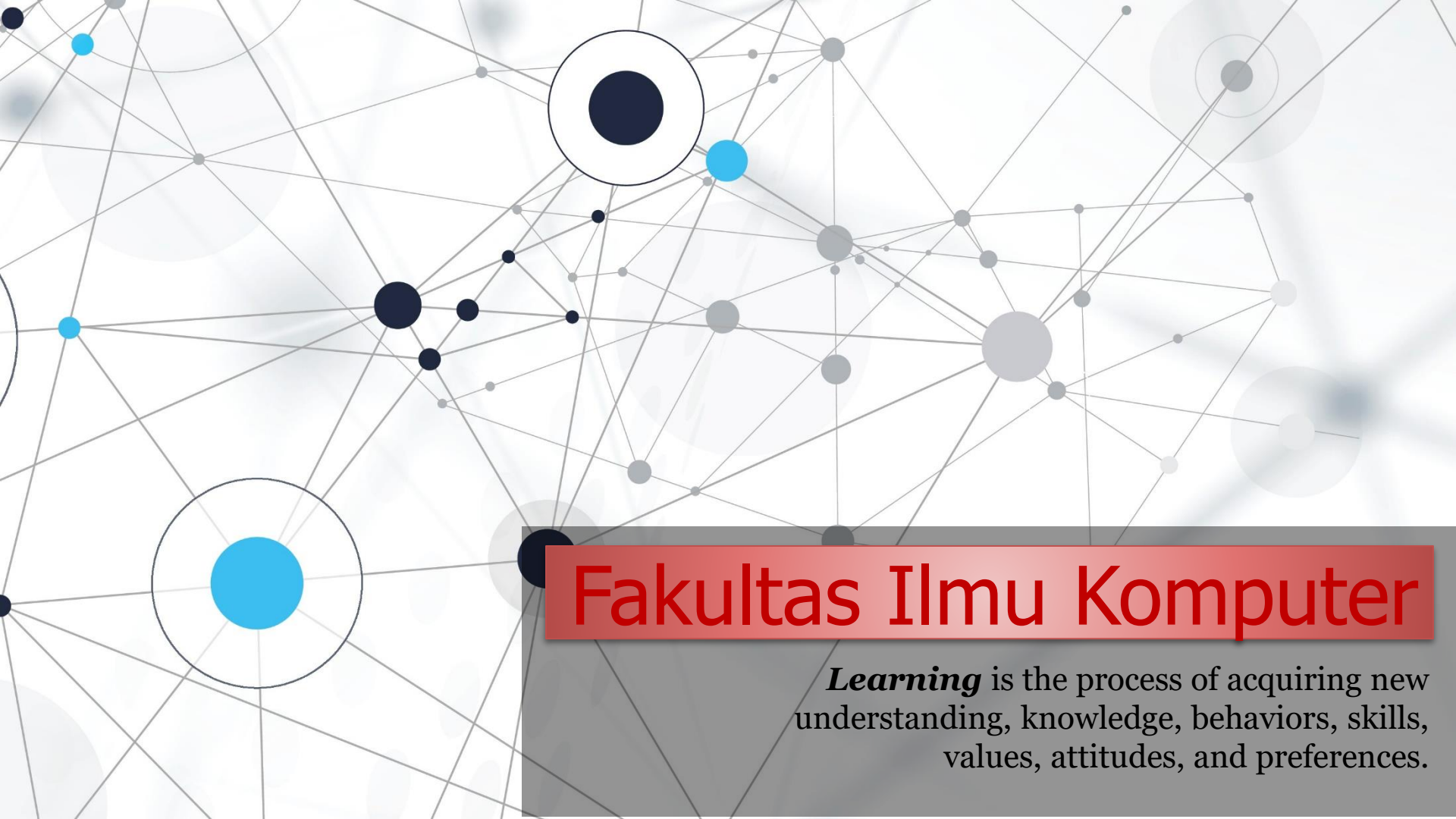


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

# Fakultas Ilmu Komputer

***Learning*** is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

PHP MVC (*model, view, controller*)

## Framework #2 – Structure & URL Routing



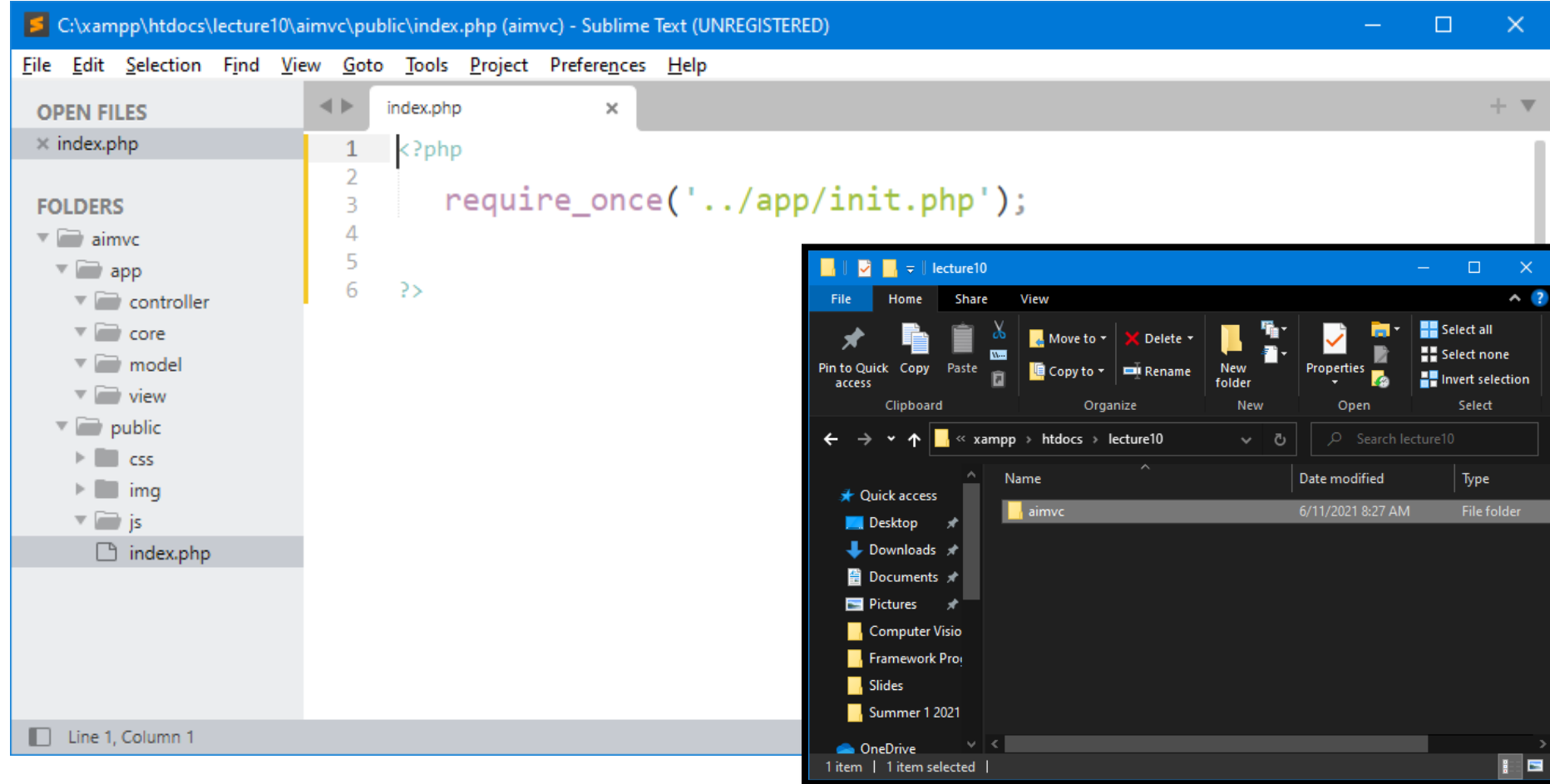
A decorative border of green leaves and branches at the top of the slide.

# 1 - File & Directory Structures

A decorative border of green leaves and branches at the bottom of the slide.

# Basic File & Directory Structures

**Directory Path:** `C:/xampp/htdocs/lecture10/aimvc/`



**Main directory of our new framework:**  
**aimvc** ⇒ Artificial Intelligence Model View Controller ⇒ PHP Framework

# Create Framework Structure

| No | Directory/File | Description ( <i>optional</i> )                                                                                                       |
|----|----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1  | app            | app directory is the web application's <b>private directory</b> of this new framework.                                                |
| 2  | public         | <b>public directory</b> is to store all the public static resources like <b>html</b> , <b>css</b> , <b>image</b> and <b>js</b> files. |
| 3  | core           | core directory is to store all files for routing and it will store the framework's core classes.                                      |
| 4  | model          | model directory is for all app's Model classes.                                                                                       |
| 5  | view           | view directory is for all app's View classes.                                                                                         |
| 6  | controller     | controller directory is for all app's Controller classes.                                                                             |
| 7  | css            | css directory is for Cascading Style Sheets (CSS) files.                                                                              |
| 8  | img            | Img directory is for images files.                                                                                                    |
| 9  | js             | js directory is for JavaScript files.                                                                                                 |
| 10 | index.php      | index.php file is the 'entry-point' file, the front controller to the users.                                                          |

## Additional Directories:

**template directory** stores the template files like *header* file and *footer* file.

**config directory** stores the app's configuration files.

**upload directory** is for uploaded files, such as uploaded images.

**db directory** is database related classes, such as database driver classes.

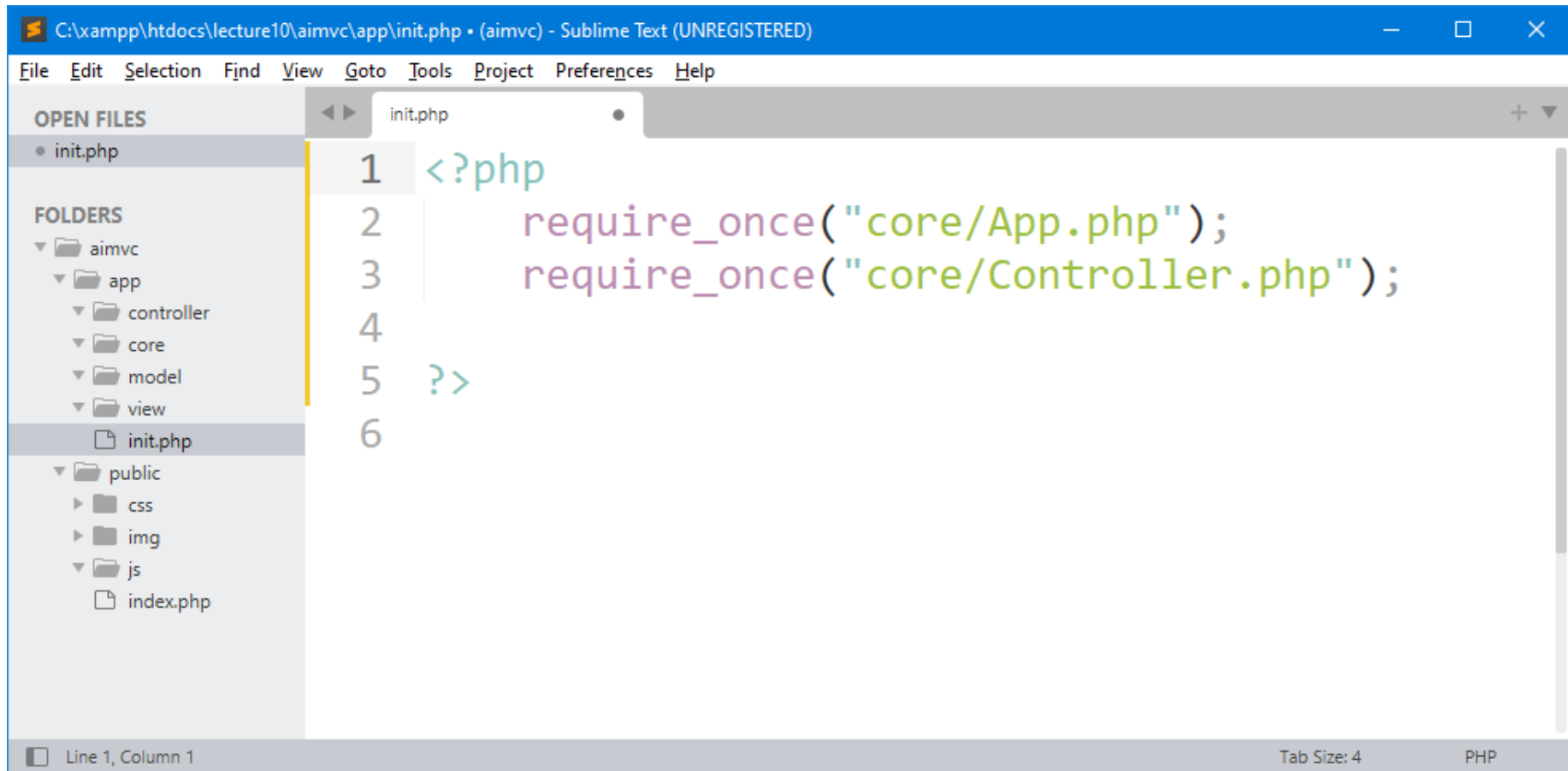
**helper directory** is help/assistant functions.

**library directory** is for class libraries.

# Create The file *init.php*

init.php (*initialization*) is an optional file within our Framework file structure and located in *private directory*. This file may contain the initialization of event handlers and connection of additional functions that are common for all websites.

**File Path:** `C:/xampp/htdocs/lecture10/aimvc/app/` (*private directory*)



```
C:\xampp\htdocs\lecture10\aimvc\app\init.php • (aimvc) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
• init.php

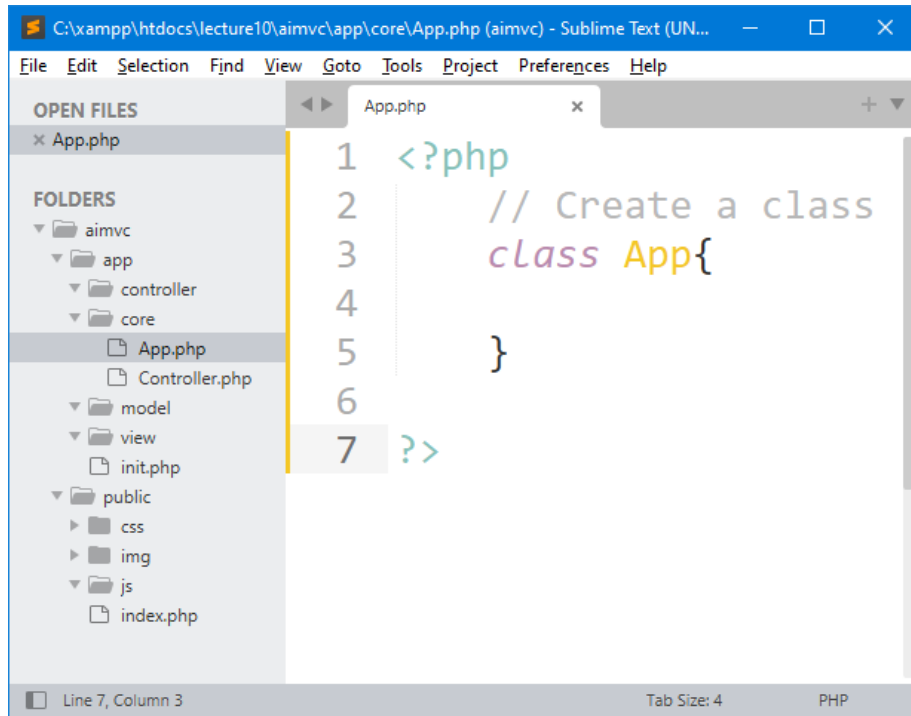
FOLDERS
▼ aimvc
  ▼ app
    ▼ controller
    ▼ core
    ▼ model
    ▼ view
    init.php
  ▼ public
    ► css
    ► img
    ▼ js
    index.php

1 <?php
2     require_once("core/App.php");
3     require_once("core/Controller.php");
4
5 ?>
6

Line 1, Column 1 Tab Size: 4 PHP
```

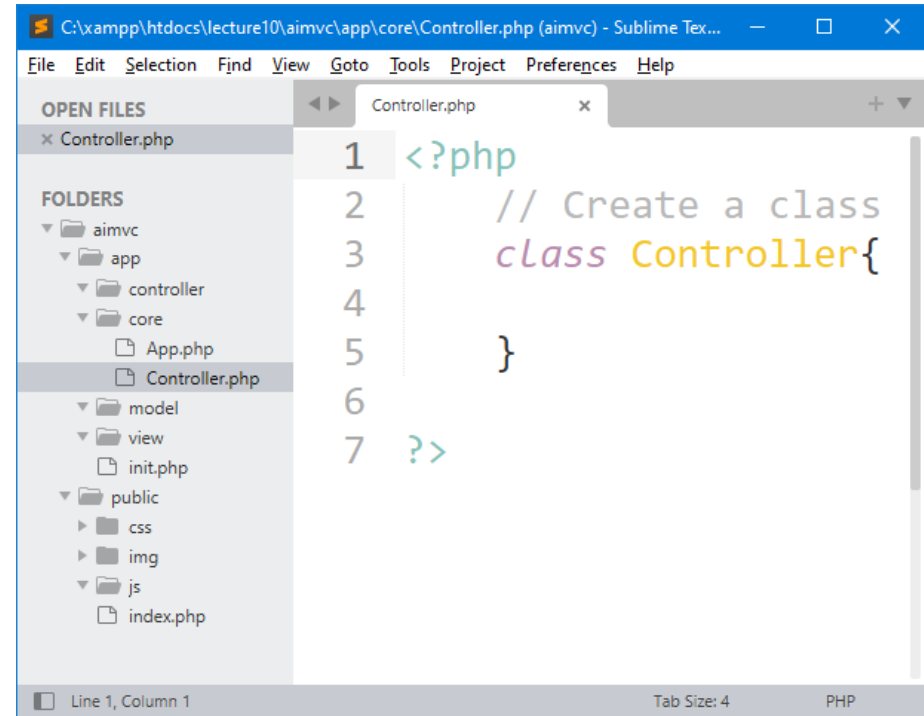


# Create Framework's Core Classes



```
1 <?php
2 // Create a class
3 class App{
4
5 }
6
7 ?>
```

The screenshot shows the Sublime Text editor with the file 'App.php' open. The left sidebar displays a file explorer with the following structure: aimvc > app > controller > core > App.php. The main editor area shows the PHP code for the 'App' class, which is currently empty except for the class declaration and closing tag. The status bar at the bottom indicates 'Line 7, Column 3'.



```
1 <?php
2 // Create a class
3 class Controller{
4
5 }
6
7 ?>
```

The screenshot shows the Sublime Text editor with the file 'Controller.php' open. The left sidebar displays a file explorer with the following structure: aimvc > app > controller > core > Controller.php. The main editor area shows the PHP code for the 'Controller' class, which is currently empty except for the class declaration and closing tag. The status bar at the bottom indicates 'Line 1, Column 1'.

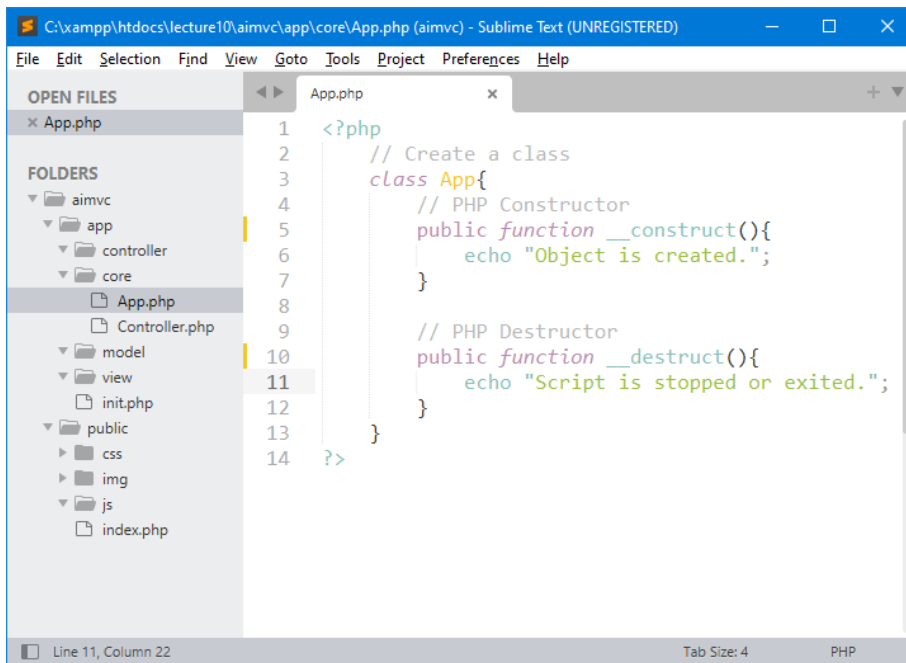
Class App (*App.php*) digunakan sebagai class utama dalam pembentukan aimvc framework. Class Controller (*Controller.php*) digunakan sebagai class yang akan mengatur semua fungsi-fungsi yang akan diletakan di dalam *controller directory*.

## Note:

First letter as a capital letter (*uppercase letter*) normally means a php class.

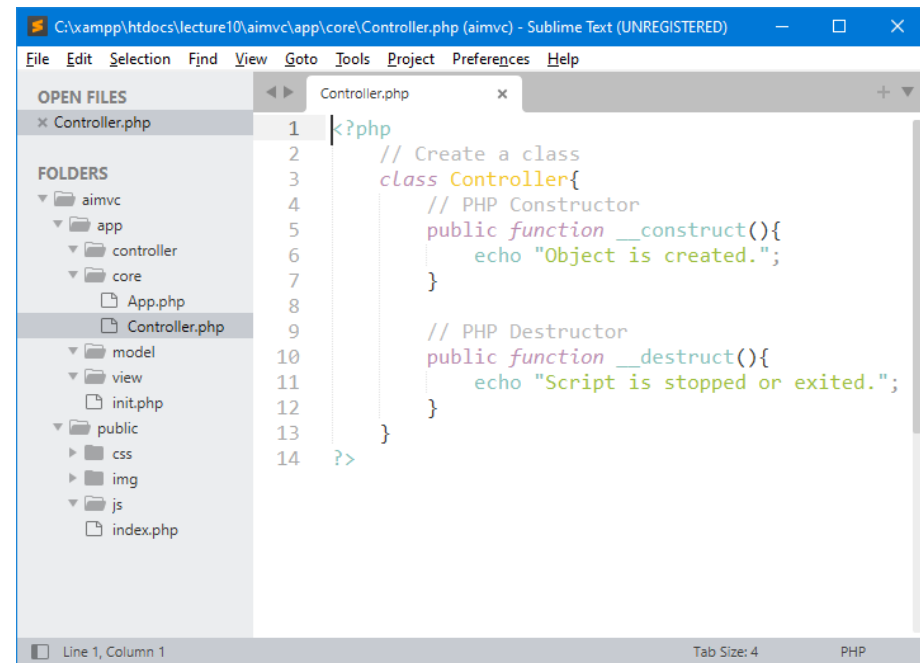
# Constructors and Destructors

Tambahkan fungsi **\_\_construct** dan **\_\_destruct** di Class App (**App.php**) dan Class Controller (**Controller.php**), ini berfungsi agar supaya pada saat proses instansiasi atau proses pembuatan instance/objek dari class App (**App.php**) dan class Controller (**Controller.php**) selesai, maka sudah bisa langsung di *display* contoh output-nya.



The screenshot shows the Sublime Text editor with the file 'App.php' open. The code defines a class 'App' with a constructor and a destructor. The file explorer on the left shows the project structure with folders like 'aimvc', 'app', 'controller', 'core', 'model', 'view', 'public', 'css', 'img', 'js', and 'index.php'.

```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6         echo "Object is created.";
7     }
8
9     // PHP Destructor
10    public function __destruct(){
11        echo "Script is stopped or exited.";
12    }
13 }
14 ?>
```



The screenshot shows the Sublime Text editor with the file 'Controller.php' open. The code defines a class 'Controller' with a constructor and a destructor. The file explorer on the left shows the project structure with folders like 'aimvc', 'app', 'controller', 'core', 'model', 'view', 'public', 'css', 'img', 'js', and 'index.php'.

```
1 <?php
2 // Create a class
3 class Controller{
4     // PHP Constructor
5     public function __construct(){
6         echo "Object is created.";
7     }
8
9     // PHP Destructor
10    public function __destruct(){
11        echo "Script is stopped or exited.";
12    }
13 }
14 ?>
```

## PHP - The **\_\_construct** Function:

A constructor allows you to initialize an object's properties upon creation of the object.

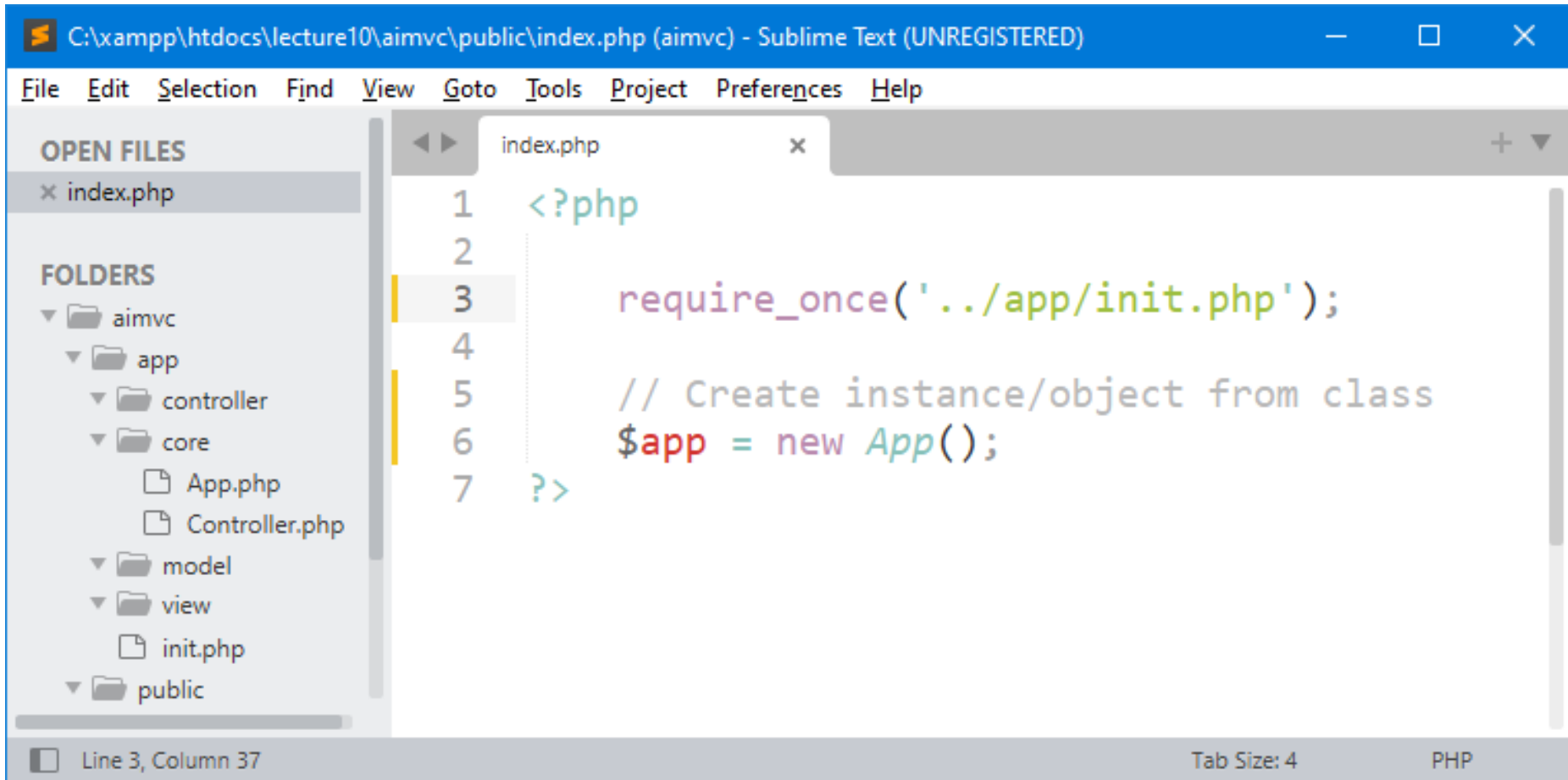
## PHP - The **\_\_destruct** Function:

A destructor is called when the object is destructed or the script is stopped or exited.



# Instansiasi Class (Object Class) App

Instansiasi **class App** (*App.php*) dari file **index.php** adalah pembuatan instance/objek dari *class App*, dimana kita akan coba menampilkan/view sesuatu dari *class* tersebut. Perhatikan di dalam *constructor* dari *class App* sudah terdapat *statement echo* (*which is used to display the output*) dimana akan digunakan untuk menampilkan *output* sederhana.



```
C:\xampp\htdocs\lecture10\aimvc\public\index.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x index.php

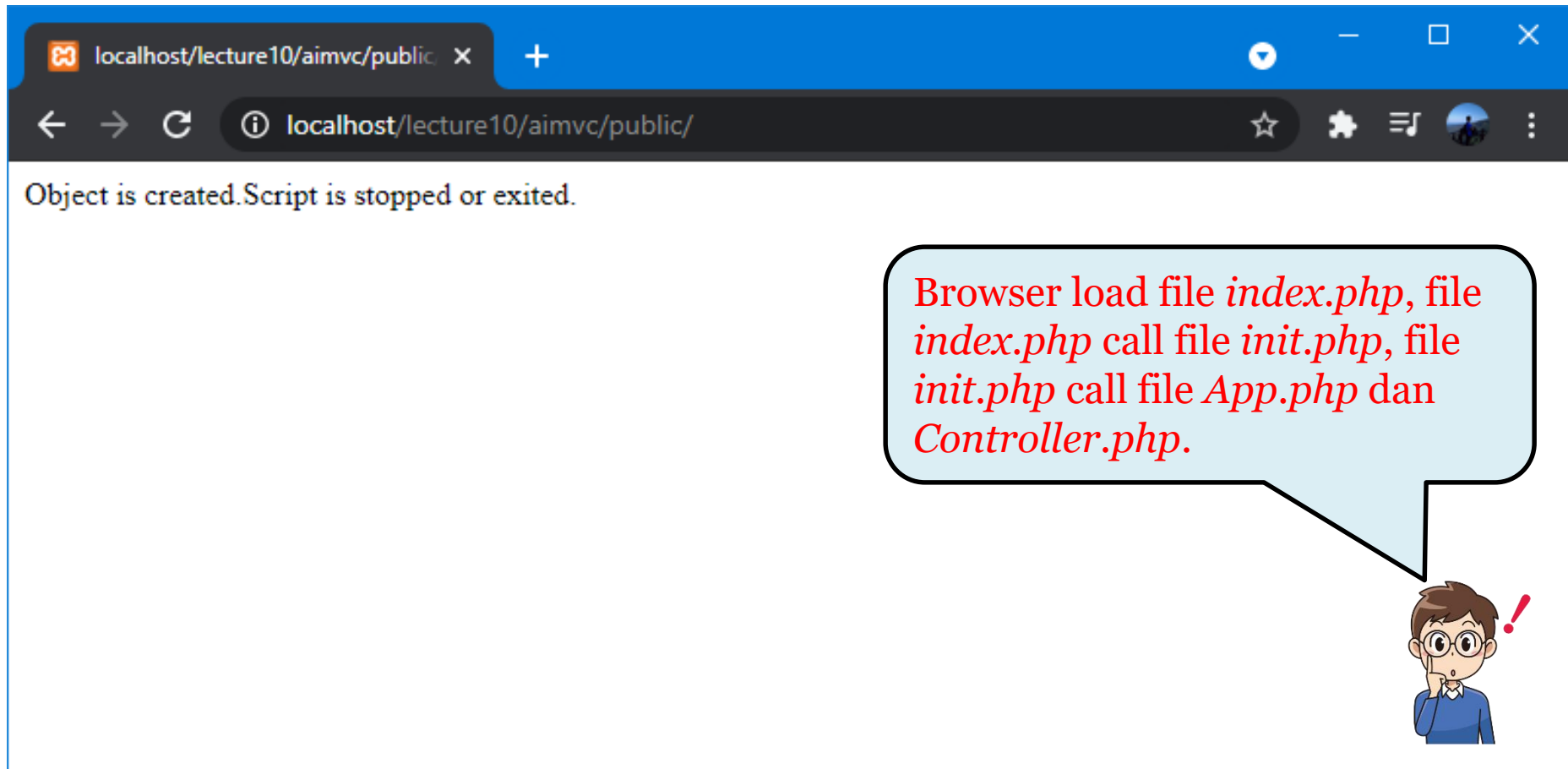
FOLDERS
aimvc
  app
    controller
    core
      App.php
      Controller.php
    model
    view
      init.php
    public

1 <?php
2
3     require_once(' ../app/init.php');
4
5     // Create instance/object from class
6     $app = new App();
7 ?>
```

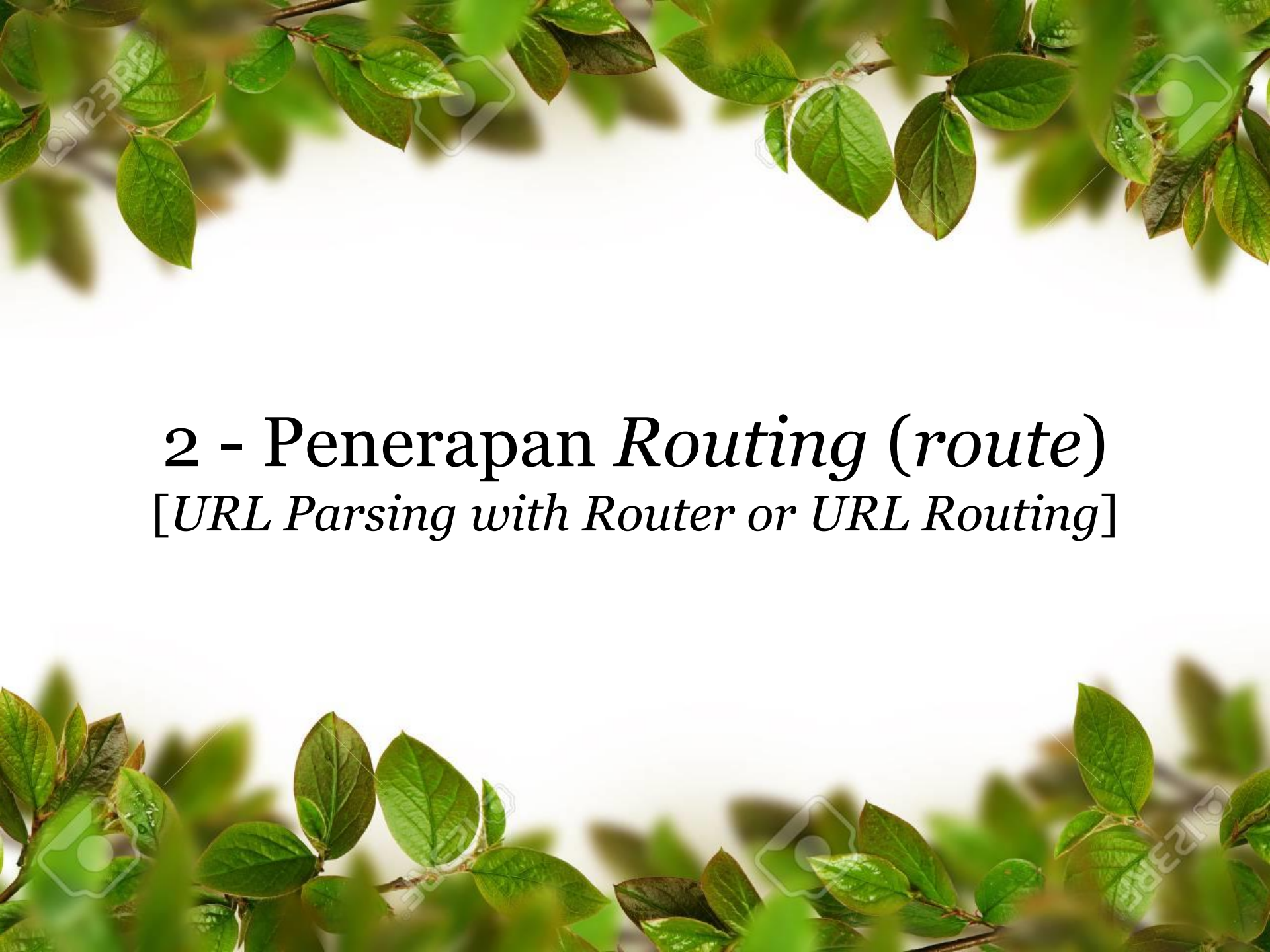
Line 3, Column 37 Tab Size: 4 PHP

# Test *your* Framework's Structure

Uniform Resource Locator (URL): <http://localhost/lecture10/aimvc/public/>



Pada saat jalankan (*run*) PHP Files di *browser*, maka *output* yang diharapkan (*giving the expected output*) yaitu seperti diatas.



## 2 - Penerapan *Routing* (*route*)

[*URL Parsing with Router or URL Routing*]

# Istilah/Term Routing

- ❑ **Routing** adalah proses dimana suatu *item* dapat sampai ke tujuan dari satu lokasi ke lokasi lain. Dalam hal ini, *item* yang dimaksud adalah halaman aplikasi website. Para *developer* dapat menentukan sendiri halaman yang akan muncul pada saat dikunjungi oleh *users*.
- ❑ **Routing** is the process of selecting a path for traffic in a network or between or across multiple networks. Broadly, routing is performed in many types of networks, including circuit-switched networks, such as the public switched telephone network (PSTN), and computer networks, such as the Internet. - Wikipedia

## Contoh penggunaan Routing:

Pada saat *user* mengunjungi halaman *dashboard*, maka *kita* dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.

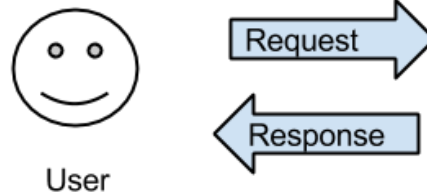
**Route** dapat handle semua perintah yang telah dideklarasikan atau bahkan **route** bisa dianalogikan sebagai peta petunjuk bagaimana alur navigasi aplikasi yang sedang dibangun atau juga bisa disebut *Router* merupakan suatu rute/jalan/trayek.



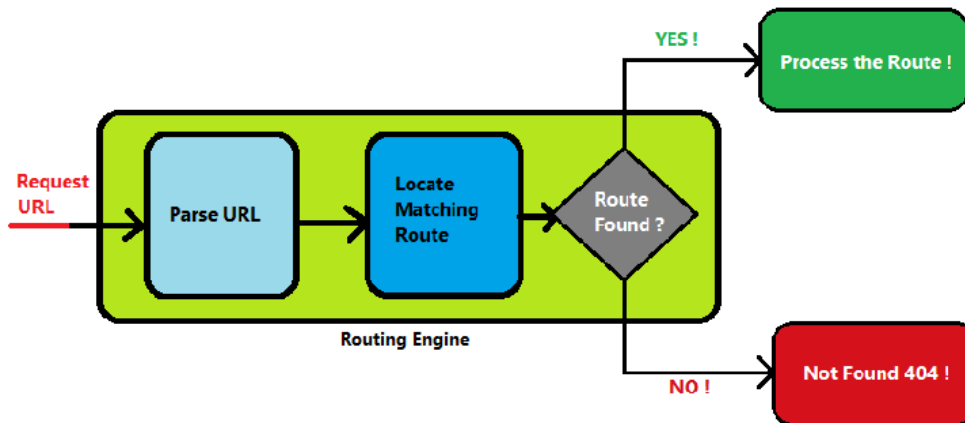
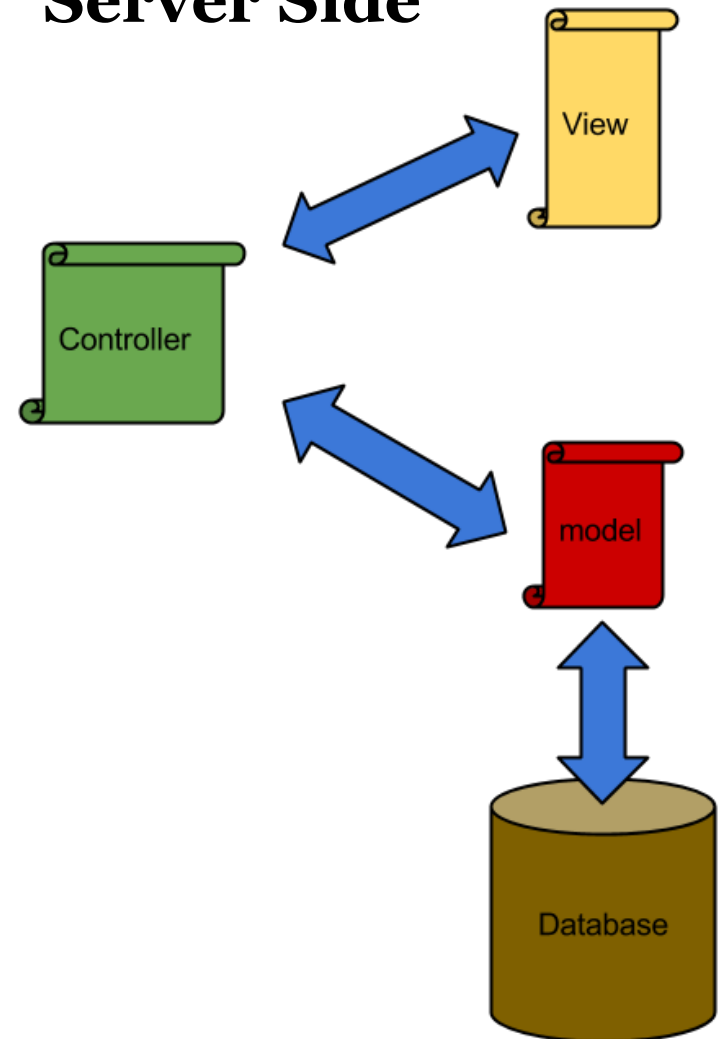
# Framework Routing

Pada saat *user* mengunjungi halaman *home*, maka kita dapat menentukan tampilan apa yang akan muncul, apakah itu hanya berupa tulisan (*text*), berupa halaman *controller*, berupa halaman *view*, maupun halaman *error*.

## Client Side

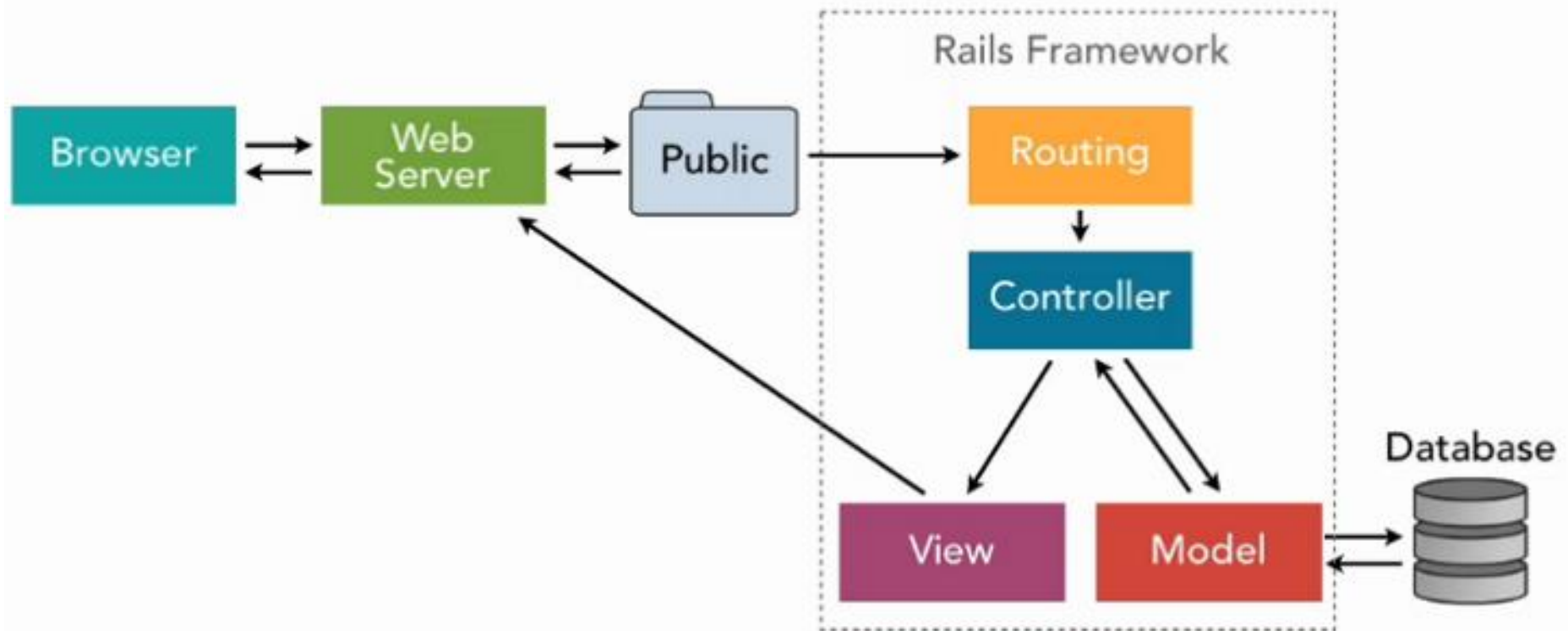


## Server Side



# Rails Architecture

**Ruby on Rails**, or **Rails**, is a server-side web application framework written in Ruby under the MIT License. Rails is a ***model–view–controller*** (MVC) framework, providing default structures for a database, a web service, and web pages.





# URL Routing

Typically there is a one-to-one relationship (*hubungan satu-ke-satu*) between a URL string and its corresponding controller class/method. The segments in a URL (*bagian dalam URL*) normally follow this pattern:

`http://www.example.com/class/method/parameter1/parameter2/`

## What is a Controller?

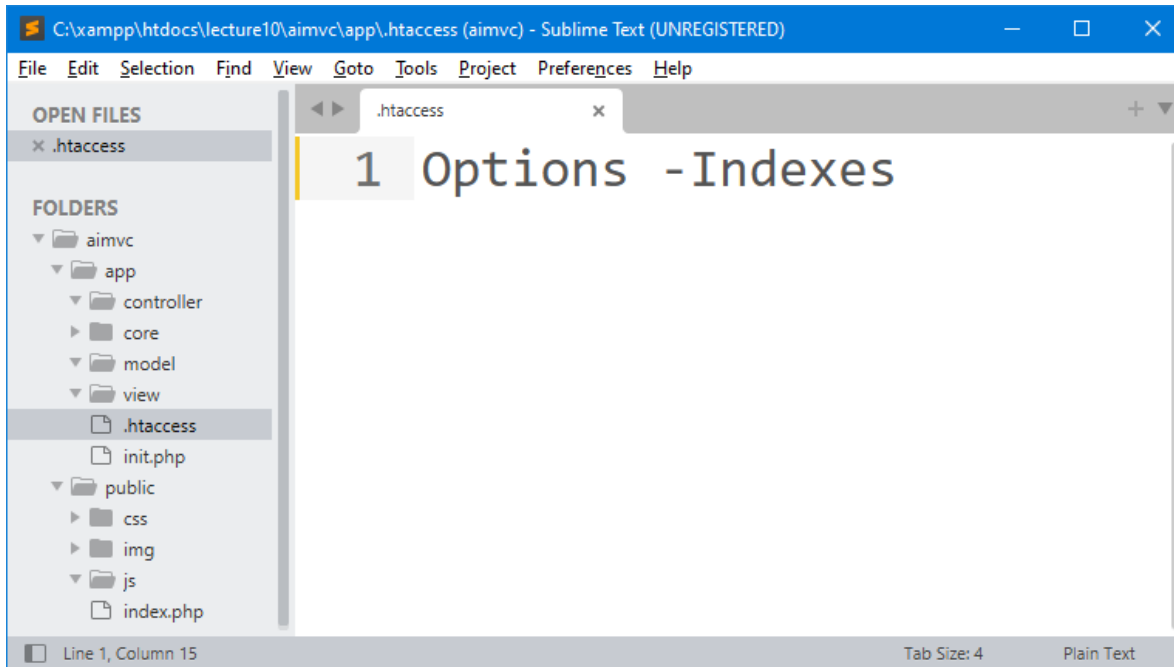
A Controller is simply a class file that is named in a way that can be associated with a URI.

**Class** merupakan *controller class* yang sudah dibuat dan biasanya segmen kedua (**method**) dari URL disiapkan untuk nama *metode*.

# Setting *your* own routing rules

# 1 - Restrict Users Access (*private*)

Block/restrict access permission to ***private*** directory (*app directory*):

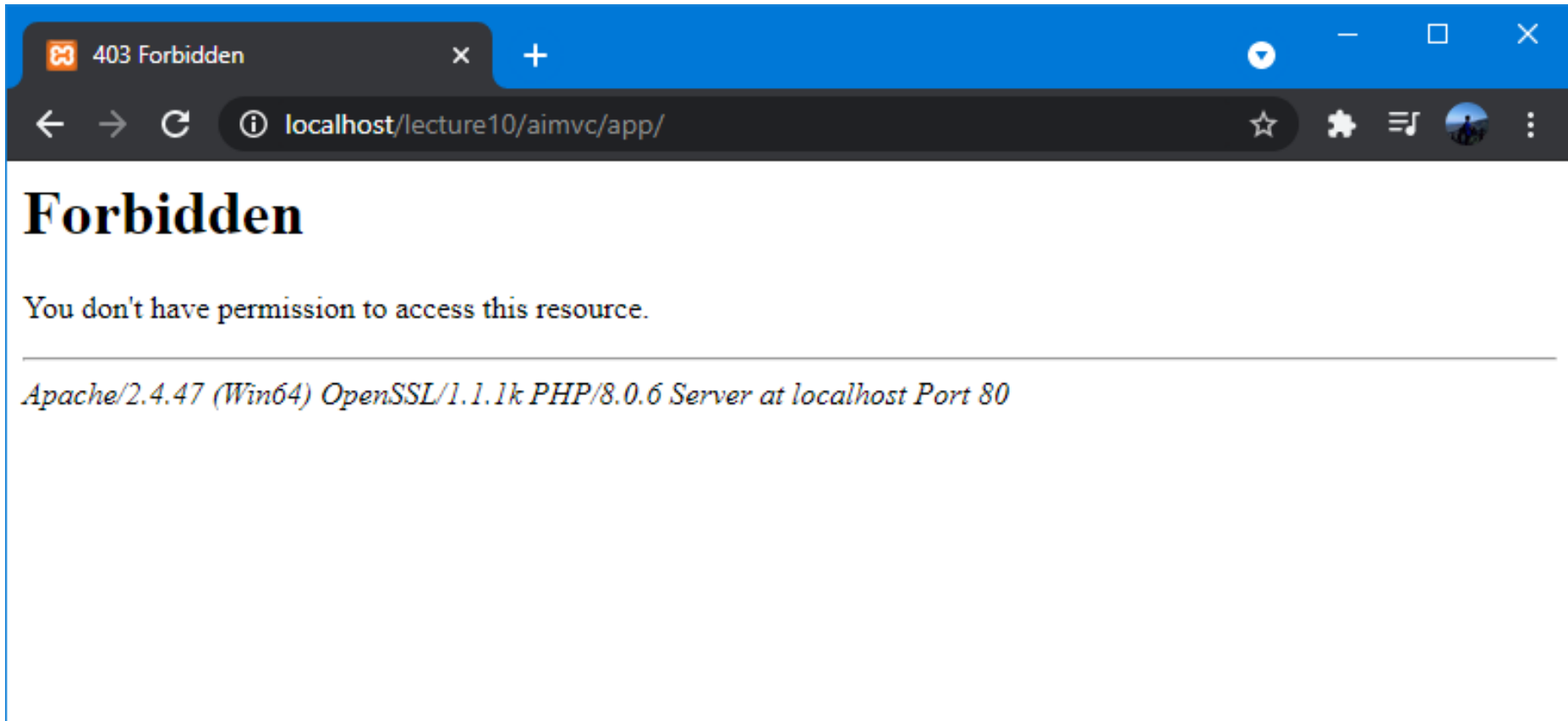


Create file **.htaccess** di dalam folder app. “**Options –Indexes**” artinya jika ada *users* membuka folder app dan folder-folder yang lain di dalamnya maka jangan tampilkan isi folder-nya, selama di dalam *folder* tersebut tidak ada file *index*. Dalam arti user tidak dapat diberi izin.

File **.htaccess** adalah cara untuk mengkonfigurasi rincian website kita tanpa perlu mengubah *file konfigurasi server*.

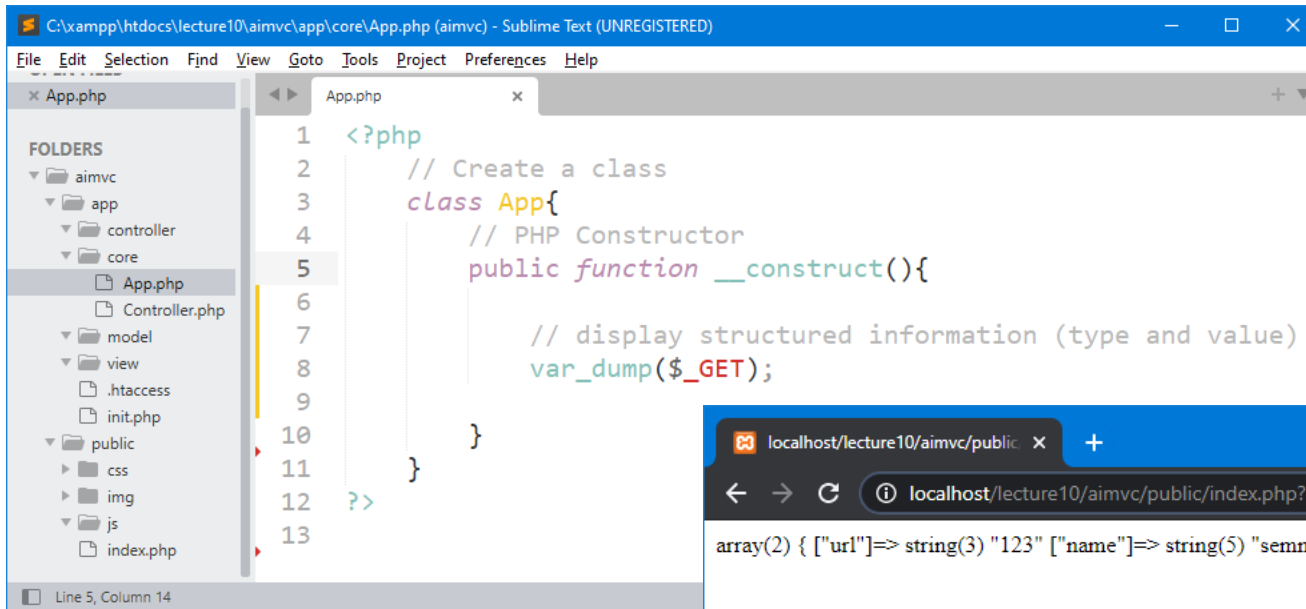
# 1 - Restrict Users Access (*private*)

Test apakah *users* masih diberi *access* permission ke ***private*** directory (*app directory*) dan folder-folder yang lain di dalamnya? Jika iya, maka dapat mengaktifkan *.htaccess* file pada *httpd.conf* file.



Test dibuka juga directory <http://localhost/lecture10/aimvc/app/view/>

# 2 - Test Request URL Public Directory

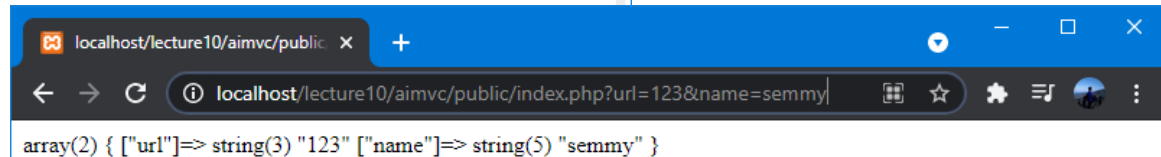


The screenshot shows a Sublime Text editor window titled "C:\xampp\htdocs\lecture10\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)". The left sidebar displays a file explorer with the following structure:

- aimvc
  - app
    - controller
    - core
      - App.php**
      - Controller.php
  - model
  - view
  - .htaccess
  - init.php
  - public
    - css
    - img
    - js
    - index.php

The main editor area shows the content of App.php:

```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6
7         // display structured information (type and value)
8         var_dump($_GET);
9
10    }
11 }
12 ?>
13
```



The screenshot shows a web browser window with the address bar displaying "localhost/lecture10/aimvc/public/index.php?url=123&name=semmy". The browser's developer tools console shows the output of the PHP script:

```
array(2) { ["url"]=> string(3) "123" ["name"]=> string(5) "semmy" }
```

## Request URL:

<http://localhost/lecture10/aimvc/public/index.php?url=123&name=semmy>

**Jika dilihat URL yang di *request* tidak teratur, oleh sebab itu kita akan mencoba Parsing URL (*mengurai URL*) dan membuatnya lebih rapi sesuai dengan *standard framework* yang akan di *design*.**

# Apply PHP isset() Function in URL

We used *isset function* to check whether a variable (*get url*) is empty and also check whether the variable (*get url*) is set/declared before setting our own routing rules.

## ***Example isset function:***

```
<!DOCTYPE html>
<html>
<body>
  <?php
    $a = 0;
    // True because $a is set
    if (isset($a)) {
      echo "Variable 'a' is set.<br>";
    }

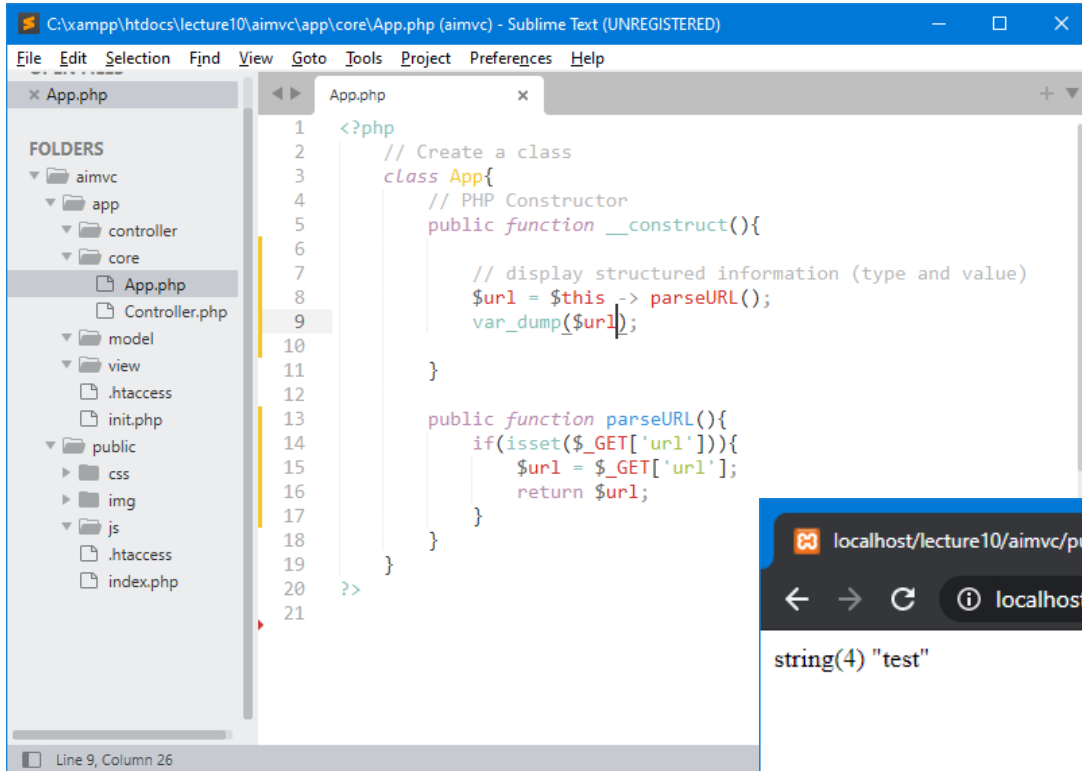
    $b = null;
    // False because $b is NULL
    if (isset($b)) {
      echo "Variable 'b' is set.";
    }
  ?>
</body>
</html>
```

|                       |                                                                 |
|-----------------------|-----------------------------------------------------------------|
| <b>Return Value:</b>  | TRUE if <i>variable</i> exists and is not NULL, FALSE otherwise |
| <b>Return Type:</b>   | Boolean                                                         |
| <b>PHP Version:</b>   | 4.0+                                                            |
| <b>PHP Changelog:</b> | PHP 5.4: Non-numeric offsets of strings now returns FALSE       |

The `isset()` function checks whether a variable is set, which means that it has to be declared and is not NULL. This function returns true if the variable exists and is not NULL, otherwise it returns false.

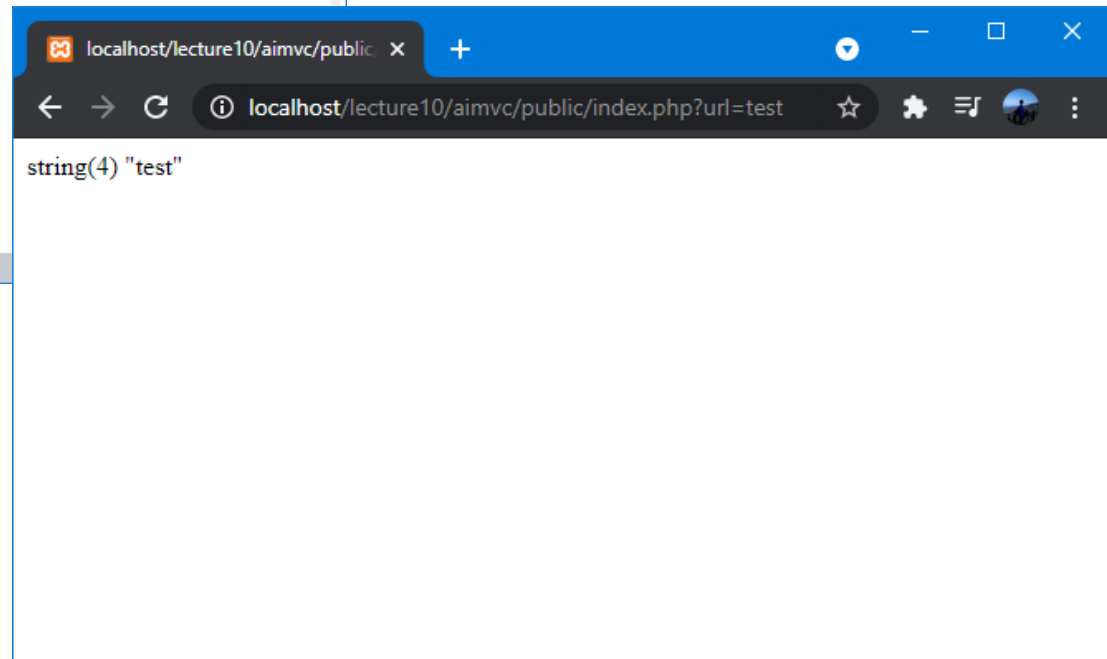


# 3 – Process Parsing URL



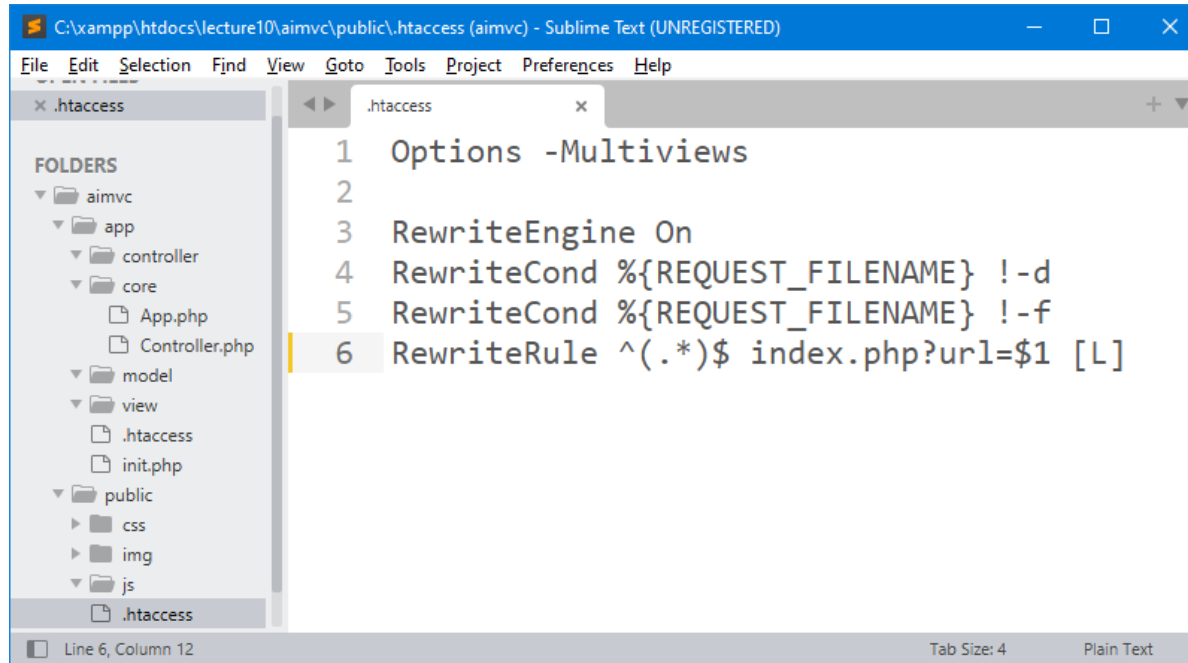
```
1 <?php
2 // Create a class
3 class App{
4     // PHP Constructor
5     public function __construct(){
6
7         // display structured information (type and value)
8         $url = $this->parseURL();
9         var_dump($url);
10
11     }
12
13     public function parseURL(){
14         if(isset($_GET['url'])){
15             $url = $_GET['url'];
16             return $url;
17         }
18     }
19 }
20 ?>
21
```

Line 9, Column 26



# 4 - Avoid Users Access (*public*)

Menghindari *kesalahan access file* atau **public** folder (*public directory*).



The screenshot shows a Sublime Text window titled "C:\xampp\htdocs\lecture10\aimvc\public\.htaccess (aimvc) - Sublime Text (UNREGISTERED)". The left sidebar displays a file explorer with the following structure:

- aimvc
  - app
    - controller
  - core
    - App.php
    - Controller.php
  - model
  - view
    - .htaccess
    - init.php
  - public
    - css
    - img
    - js
    - .htaccess

The main editor area shows the content of the selected .htaccess file:

```
1 Options -Multiviews
2
3 RewriteEngine On
4 RewriteCond %{REQUEST_FILENAME} !-d
5 RewriteCond %{REQUEST_FILENAME} !-f
6 RewriteRule ^(.*)$ index.php?url=$1 [L]
```

The status bar at the bottom indicates "Line 6, Column 12", "Tab Size: 4", and "Plain Text".

Create file **.htaccess** di dalam folder public. “**Options - Multiviews**” artinya kita akan menghindari kesalahan akses file ataupun directory dari URL (*Uniform Resource Locator*).

Karena *users* akan mengakses langsung *file* class bukan ke *folder* yang ada di **public directory** melalui URL. Maka kesalahan akses ataupun **SQL Injection techniques** (*backend database manipulation to access information*) dapat kita hindari.

## 4 - Avoid Users Access (*public*)

```
RewriteCond %{REQUEST_FILENAME} !-d  
RewriteCond %{REQUEST_FILENAME} !-f
```

Rewrite condition (**RewriteCond**) tests along with a corresponding rule (**RewriteRule**) if the prior conditions pass.

Jika URL yang di *request* adalah *directory/folder* (*d*) dan *file* (*f*) maka akan diabaikan (*ignored*). Dalam arti, *request* yang diinginkan hanyalah berupa *method*.

# 4 - Avoid Users Access (*public*)

## RewriteRule **^(.\*)\$ index.php?url=\$1 [L]**

RewriteRule defines a particular rule.

### Fungsi Regex **^(.\*)\$** artinya :

1. Symbol caret ( **^** ) artinya yang ditulis mulai dari awal url (*match the beginning of a line*) setelah penulisan url “/aimvc/public/”.
2. in regular expressions, symbol dot ( **.** ) is a special character used to match any one character. Jadi, akan ambil semua character sesudah “aimvc/public”.
3. A regular expression followed by an asterisk ( **\*** ) matches zero or more occurrences of the regular expression. Jadi ambil character satu persatu dari url.
4. If a **dollar sign** ( **\$** ) is at the end of the entire **regular expression**, it matches the end of a line. Jadi akan dicek sampai semua character (*text*) yang ada dalam url selesai.

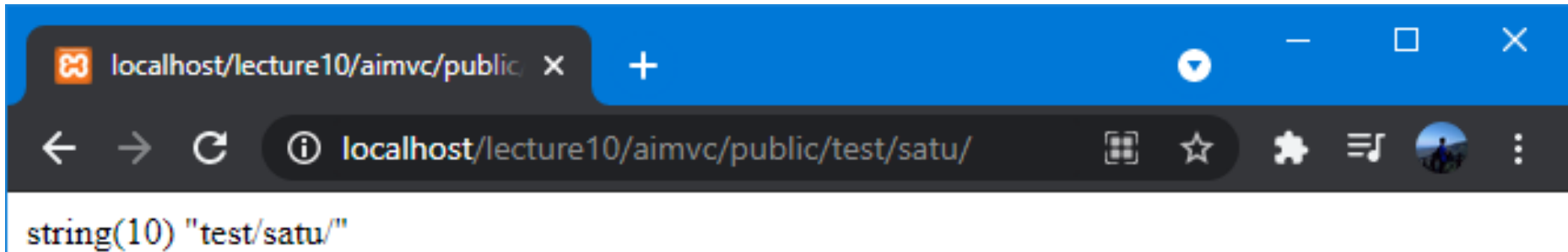
### Fungsi Regex **index.php?url=\$1** artinya:

Arahkan ke *file* index.php yang mengirimkan url dan disimpan di \$1 (*placeholder*).

### Flag **[L]** artinya:

- ☐ The [L] flag causes *mod\_rewrite* to stop processing the rule set.
- ☐ In most contexts, this means that if the rule matches, *no further rules will be processed*.

# Request URL without Format



**Langkah awal untuk *Request URL, parsing URL* dan dapatkan controller class & method dengan memanfaatkan *.htaccess file* success !!!**



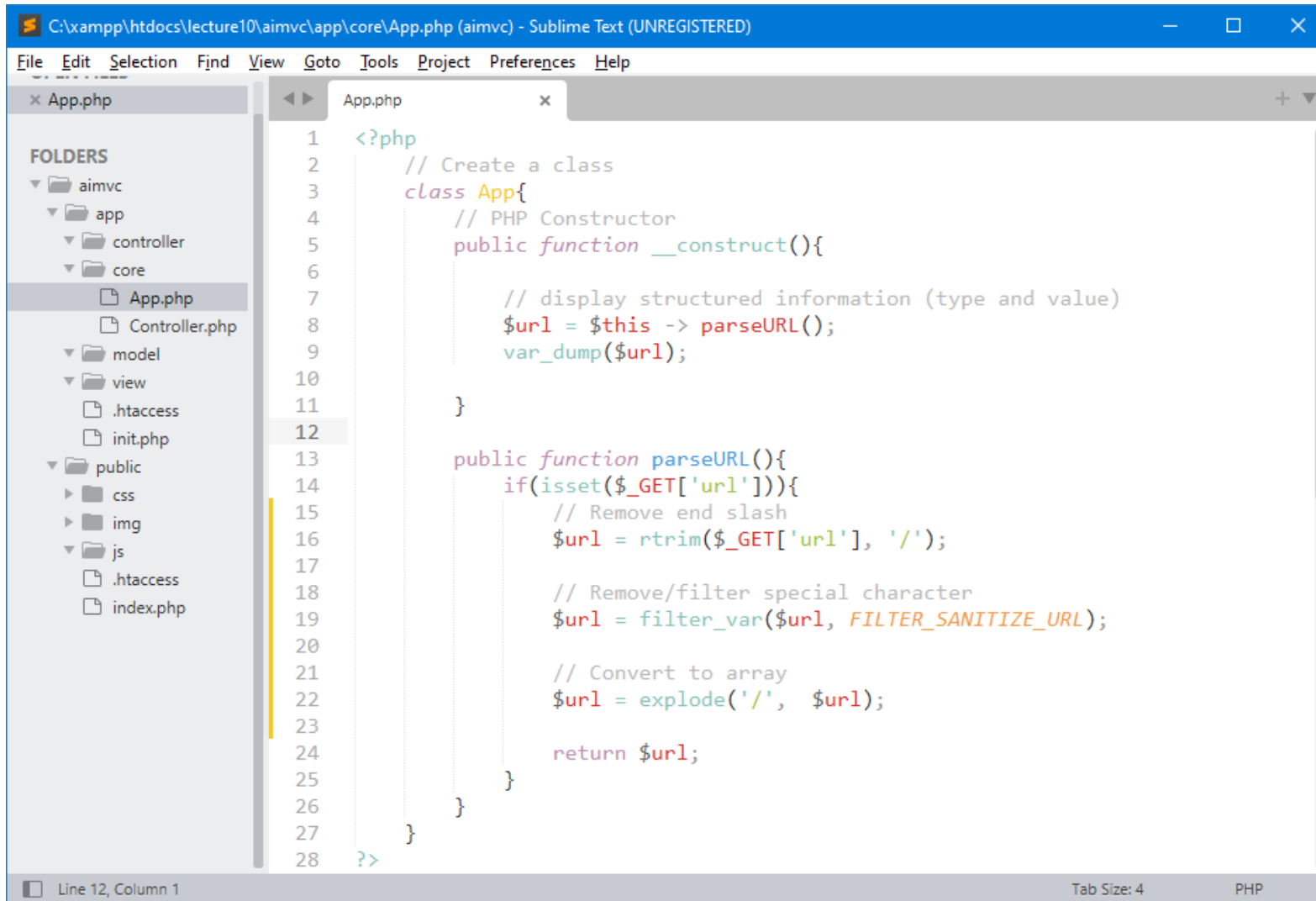
## Setting *your* own routing rules

3 – Remove end “/” in Url, Special Character and Covert to Array.



# Handling Request URL

Handle more *routing rules* when *users* try to request through URL.



```
C:\xampp\htdocs\lecture10\aimvc\app\core\App.php (aimvc) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

x App.php
FOLDERS
  aimvc
    app
      controller
      core
        App.php
        Controller.php
      model
      view
        .htaccess
        init.php
    public
      css
      img
      js
        .htaccess
        index.php

1  <?php
2      // Create a class
3      class App{
4          // PHP Constructor
5          public function __construct(){
6
7              // display structured information (type and value)
8              $url = $this -> parseURL();
9              var_dump($url);
10
11          }
12
13      public function parseURL(){
14          if(isset($_GET['url'])){
15              // Remove end slash
16              $url = rtrim($_GET['url'], '/');
17
18              // Remove/filter special character
19              $url = filter_var($url, FILTER_SANITIZE_URL);
20
21              // Convert to array
22              $url = explode('/', $url);
23
24              return $url;
25          }
26      }
27  }
28  ?>
```

Line 12, Column 1      Tab Size: 4      PHP

# Test Router - Request URL







exercise for students

# Latihan untuk *Students*

**Create directory** : C:\xampp\htdocs\lecture11\  
**Create file** : \_\_.php

Buat semua contoh yang sudah di pelajari sebelumnya.

# END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

